

Vortex ring dynamic in a stratified fluid at rest

1 Exigences de programmation

Dans ces projets d'informatique scientifique, vous allez devoir modifier vos jeunes habitudes de programmation pour devenir un peu plus professionnel. Les règles suivantes sont obligatoires et non négociables:

1. **Nommage:** toutes les fonctions et toutes les variables doivent avoir un nom expressif, c'est-à-dire qu'un lecteur quelconque du programme doit comprendre l'utilisation ou le sens. Il y a une tolérance sur les variables qui ont un sens mathématique ou physique commun comme x , y , t , i , j , π , r

Il est déconseillé d'utiliser des lettres majuscules mais ce sera toléré cette année.

2. **Commentaires:** chaque début de script ou de fonctions contient une ou plusieurs lignes de commentaires indiquant à minima les valeurs retournées (outputs) et les paramètres d'entrée (inputs): nom, type, sens.

Exemple % x: array of real, abscissa

types: real, integer, string, boolean, array of ...

3. **Langues:** les commentaires, les noms de variables, ode fonctions et du script sont en anglais.

4. **Gestion des lignes blanches:** une seule ligne blanche est introduite entre deux fonctions si elles sont dans le même fichier. Jamais plus d'une ligne blanche à la suite.

5. **Principe de séparation calculer-fixer de afficher - dessiner:**

On peut mettre des affichages sauvages pour le développement mais ensuite, il faut les enlever et il faut suivre ce principe de base:

- on crée un fonction pour calculer des valeurs. Souvent le nom commence par `set_`
- on crée un fonction qui va afficher les valeurs à l'écran. Souvent le nom commence par `get_` mais `display_` peut convenir
- on crée un fonction qui va dessiner une ou plusieurs courbes. Faire commencer le nom par `plot_`

6. **Taille d'une fonction ou d'un script:** le nombre de lignes d'instructions autorisées est de 15 à 20 lignes. On ne compte pas les commentaires ici.

Le principe de base est que chaque fonction ou script remplit une (ou très peu) de tâche(s) précise(s).

On peut parfois mettre plusieurs instructions simple en ligne séparées par ";".

Mais la recommandation à suivre est **une ligne par instruction**.

7. **Séparation des fichiers sources, des entrées et des sorties:**

Chaque élément doit être à sa place donc dans des dossiers séparés portant des noms explicites comme `Data`, `Src`, `Sources`, `Ouputs`, `Figures`, ... Il faut donc savoir gérer les chemins d'accès.

8. **Amélioration visuelle:** pour éviter les erreurs typographiques et d'interprétation de code source, il faut suivre les indications suivantes:

- mettre un espace entre les opérateurs:
`x = a + b + c / d` est correct, `x=a+b+c/d` est incorrect
- mettre un espace après une virgule: `x(i, j) = 2 * cos(y(i, j))` est correct,
`x(i,j) = 2 * cos(y(i,j))` est incorrect
- on ne met pas d'espace avant ou après des parenthèses
- on ne dépasse pas plus de 80 caractères par ligne.
On aligne les instructions ou variables après la coupure de ligne un utilisant le symbole de continuation `...` en Matlab.
- l'indentation est obligatoire dans les structures de tests ou les boucles.
Normalement on doit aussi indenter dans les fonctions (obligatoire en Python, optionnel en Matlab)
- Les affichages doivent être formatés tous identiquement. Par exemple un message donnant un variable est : 20 caractères suivi de : , suivi de la valeur. Privilégier une valeur par ligne.
- Ne pas hésiter à créer à l'affichage des titres de sections, avec une ligne avant et après constituée de symbole = ou *, mais sans abus.
- On n'affiche jamais des tableaux longs en entier, mais juste les 5 premiers et les 5 derniers termes par exemple. On améliore toujours l'affichage de base en mettant par exemple des titres aux colonnes.

9. Pour Matlab je conseille un fichier par fonction (portant le même nom!).

10. **Simplicité de lecture des fonctions:** le nombre d'arguments d'entrée ou de sorties d'une fonction ne peut pas dépasser 3.
Si vous en avez plus, utiliser des tableaux $q = [x, y, z]$ ou des structures de type $q.x$, $q.y$, $q.z$, la variable passée étant q .
11. Nettoyer le code et démarrer l'écriture avec les règles.
12. Développer très progressivement, 5 lignes par 5 lignes par exemple en lançant le code ou la fonction à chaque fois.
13. Les informations en cas d'erreurs sont la plus part du temps très claires, alors regarder les lignes indiquées et celles autour.
14. **Pour ce projet spécifiquement:** respecter les recommandations ci-dessus, et vous devez en plus :
 - utiliser une ou des structures
 - le script principal s'appellera `main.m`
 - mettre des commentaires pour expliquer la démarche ou commenter les résultats, car il n'y a pas de rapport à écrire, le code est le rapport.
 - le code doit être lancé simplement, comme une fonction. Mettre des exemples d'utilisation en commentaire ou début du script principal:
par exemple:
`% main(10, 6, "option")` ou `% main`, en précisant les arguments s'il y en a. Le `main` peut être une fonction.
 - mettre en évidence vos connaissances du langage Matlab en utilisant les fonctions internes adéquates.
 - au moins une figure doit être sauvegardée au format png et svg.
 - au moins un tableau doit être sauvegardé au format CSV.
 - les paramètres d'entree du code doivent être dans une unique fonction qui s'appelle `inputs.m`.
 Pas de question à l'utilisateur. Il est possible d'en mettre lors de la phase de développement cependant, pour vous uniquement.

Je ne lirai même pas les projets qui n'ont pas un `main.m` ou qui visiblement ne suivent pas les instructions, ce sera 0 tout de suite.
La notation suivra les critères suivants, si le projet est lû:

Critères	Points
respect des règles de programmation	8
respect de l'algorithme et du travail demandé	8
qualité du travail, originalité, résultats	4

Je vous conseille dorénavant de coder en suivant ces règles, quelque soit le langage, cette compétence acquise sera très valorisable.

2 Délivrables

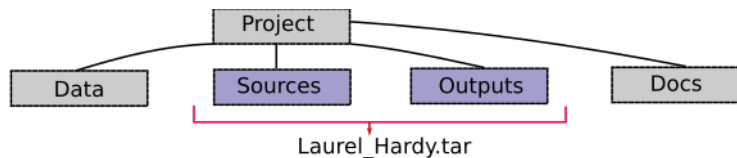


Figure 1: Architecture du projet

- La date limite d'envoi est le **lundi 8 novembre, 8h**. Mais une note, qui aura beaucoup d'importance sera mise le vendredi 29 octobre.
- La présence et les avancées au cours des séances sont aussi évaluées.
- Une absence à une séance vaut 0 pour la période d'évaluation de la séance et du jour précédent.
- Je rappelle que la note du projet comptera pour environ 50% du module "Matlab", le reste étant les évaluations des TPs et du travail en séance de TP.
- Je dois recevoir par binôme ou personne (si elle a travaillé seule) une archive au format **tar ou zip (pas rar)** dont le nom est celui du binome ou de la personne.
Il n'y aura pas de relance en cas de non suivi de cette instruction.
- Cette archive contiendra deux dossiers uniquement, le dossier "Source" et le dossier "Outputs". En aucun cas m'y coller les fichiers d'entrées qui pour votre travail devront être dans le dossier "Data", situé au même niveau que les deux dossier précédents (fig. 1).
- Ainsi en récupérant les archives et en copiant mon dossier "Data" au bonne endroit je pourrai directement faire tourner votre code.

J'ai passé du temps à écrire des consignes très précises, je n'hésiterai donc pas à mettre 5 / 20 ou moins à un projet qui ne les respectera pas, que le binôme pense avoir bien travaillé ou pas.

3 Introduction

Une partie des données expérimentales du travail de J. Albagnac [1, 2] a été récupérée. Les données sont des images déjà post-traitées qui donnent un champ de vitesse d'un anneau tourbillonnaire envoyé dans un fluide stratifié (voir fig. 2 a).

Dans le plan de coupe, on observe 2 deux zones de vorticit  positive et n gative qu'on appellera pour simplifier, *mais pas abus de langage*, un vortex positif et un vortex n gatif.

L'objet de ce travail est le suivant:

1. lire les param tres de l'exp rience
2. extraire les donn es des fichiers de vitesse
3. cr er des cartes 1D ou 2D de vitesses ou de vorticit 
4. d terminer la position et l'intensit  des deux vortex pour un temps donn 
5. d terminer le trajectoire du dip le au cours du temps
6. effectuer un travail suppl mentaire   l'initiative des  tudiants

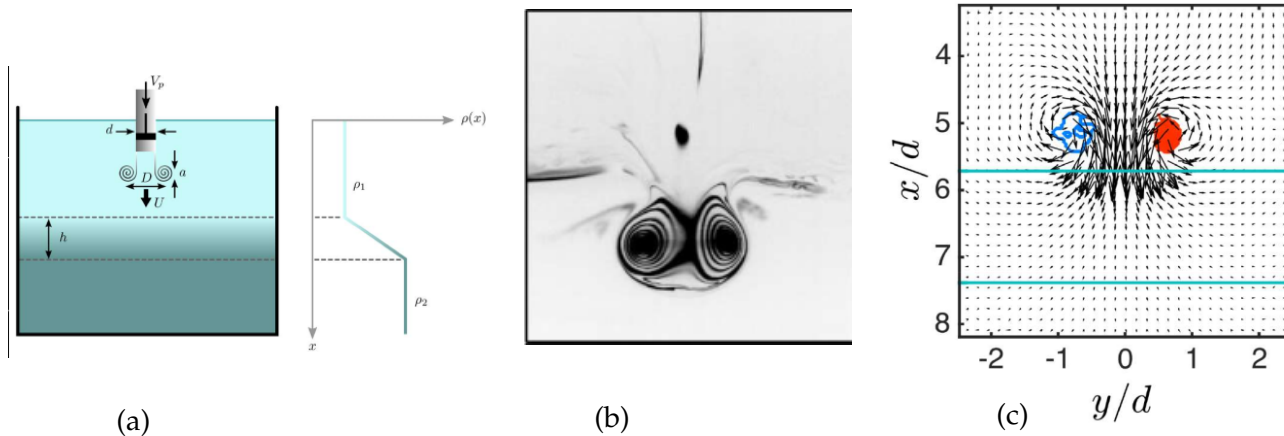


Figure 2: Vortex ring experiment and results [1], a) experimental set-up, b) example of picture, c) example of the velocity field.

4 Feuille de route (Roadmap)

Les fichiers n cessaires et les articles se trouvent sur Moodle. Il n'y a pas d'obligation ou de n cessit  de lire les articles.

Suivre les op rations ci-dessous de pr f rence dans l'ordre.

  deux, certaines peuvent  tre men es en parall le.

4.1 Lecture des param tres

Le fichier param tres s'appelle *config.mat*. Le lire et afficher correctement les param tres.

En d duire le pas de temps entre deux images Δt .

4.2 Lecture des fichiers images

Les fichiers images se nomment : *Data_00abc.mat*. Les lettres abc représentent le numéro de l'image enregistrée, de 1 à 600 et permet de définir le temps. L'espacement temporel entre deux images est Δt .

Vous ne disposez que d'une petite partie de la base de donnée qui fait plus de 3 Go.

Je conseille de démarrer le développement du code sur l'image 10 ou 50 (on note *it* ce nombre), puis de tester en dernier lieu sur des images autour des 300 pour estimer la validité de l'algorithme.

Dans les fichiers de données, les distances sont en cm et les vitesses sont en cm/s.

4.3 Exploitation d'un champ à un instant donné.

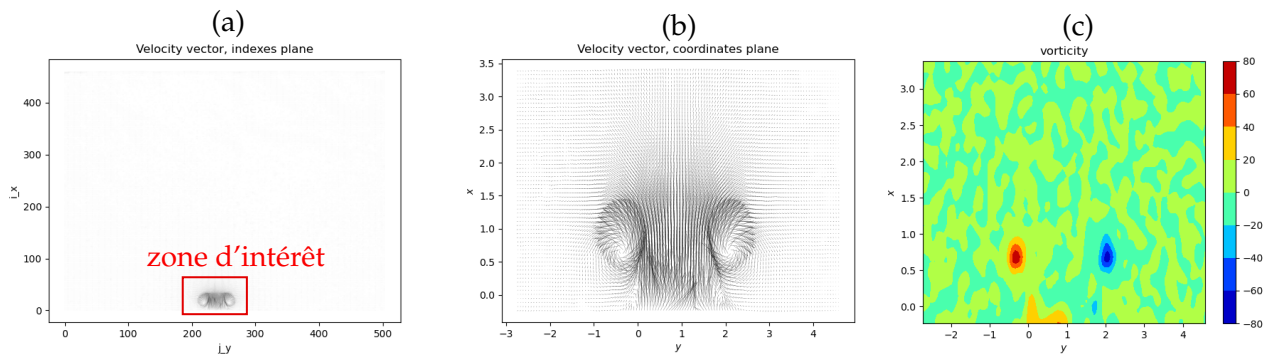


Figure 3: (a): zone d'intérêt dans le plan des indices. (b) zoom dans le plan des coordonnées. (c) vorticit . **inverser les l gendes des axes.**

Warnings

Par rapport au papier, on conserve l'axe x   l'horizontal et l'axe y   la verticale

Les actions suivantes doivent rester optionnelles pour l'utilisateur. On pourra voir l'exemple de la figure 3.

- Afficher le vecteur vitesse dans le plan (x, y) quiver.

On remarquera que l'image est invers e verticalement, l'anneau monte au lieu de descendre!

Laissez, ce n'est pas grave.

- Afficher le vecteur vitesse dans le plan (i, j) . Dans toute la suite, i est l'index pour rep rer les composantes d'un vecteur suivant x et j celle pour la composante suivant y .

  respecter dans le code: les champs lus dans les fichiers sont de la forme $u(j, i)$.

- Afficher le profil des vitesses u et v dans un plan de coupe   i constant et   j constant.

Pour le cas $it=10$, prendre $i = 219$, $j = 15$ pour tester.

- Afficher des iso-contours de u et v (contourf) dans le plan (x, y) .

L'utilisateur doit ainsi rep rer la zone d'int r t et d finir les coordonn es du cadre dans lequel il continuera de travailler $[i_{min}, i_{max}]$ et $[j_{min}, j_{max}]$.

Il peut alors relancer le calcul en indiquant ces param tres, et en r sultat on obtient les m mes choses mais en loupe. On ne s'occupe plus des endroits o  il ne se passe rien et on gagne du temps de calcul.

4.4 Calcul et exploitation de la vorticit 

- Calculer la vorticit  dans la zone d'int r t (diff rences finies d'ordre 1 ou 2, fonction `gradient`) et dessiner les iso-contours dans le plan des coordonn es et celui des indices.
On peut r p rer   l'oeil le centre de chaque vortex.
- Par le calcul du maximum et minimum, d terminer une position pr cise des vortex positif et n gatif. Utiliser une structure qui m moris  position, indices et vorticit .
- Calculer le centre du dip le, et l'intensit  du dip le sera la vorticit  du vortex positif.

4.5 Trajectoire de l'anneau tourbillonnaire

- Modifier le script afin de pouvoir lire une s quence de fichiers de donn es.
- Attention, en traitant plusieurs fichiers temporels, la zone d'int r t  volue. Penser   une modification automatique du d placement de cette zone.
- En traitant 10 fichiers temporels successifs, d terminer la position de chacun des vortex et du dip le au cours du temps.
- Tracer les trajectoires et calculer la vitesse de d placement.
- Que se passe-t-il pour les temps  lev s. Expliquer.

4.6 Premier moment de la vitesse

On fera toujours les calculs dans la zone d'int r t.

On d finit le moment de la vitesse par en consid rant un point M du domaine et l'ensemble des points P ayant une vitesse V :

$$\mathcal{M}_{Pe_z} = \sum_P \mathbf{MP} \wedge \mathbf{V}(P)$$

Si on a un vortex isol , la valeur minimale du moment se trouve lorsque le point M est le centre du vortex.

- calculer ce moment un point M de la zone d'int r t mais pour tout les points P de cette zone.
On calculera une sorte d'int grale en effectuant la sommation des moments sur l'ensemble des points P divis e par le nombre de points pr sent dans ce domaine restreint.
Cette quantit  est   conserver et   indexer dans un tableau   2 dimensions.
- Maintenant on fait varier le point M dans tout le domaine d'int r t, on obtient un nouveau tableau de valeurs qu'on nommera `value`.
- Tracer les iso-contours (`contourf`) de cette quantit  int grale. Que peut-on en conclure ? Peut-on trouver la position de l'anneau ?

4.7 Travail d'initiative

Ce travail est optionnel.

On peut trouver d'autres donn es   partir des fichiers, ou on peut chercher une meilleure m thode pour d terminer la position de l'anneau tourbillonnaire.

Ajouter des d veloppements   votre code en expliquant bien ce que vous faites.

References

- [1] Albagnac, J and Léard, Pierre and Anne-Archard, D. and Brancher, P. and Cazin, S. and Rida, Z., Dynamic of an inclined vortex ring interacting with a density stratification, Environmental Fluid Dynamics: Confronting Grand Challenges, 2019.
- [2] Pinaud, J. and Albagnac, J. and Cazin, S. and Rida, Z. and Anne-Archard, D. and Brancher, P., Three-dimensional measurements of an inclined vortex ring interacting with a density stratification.